

Digital Clock Project — Documentation

Project: 12-Hour WiFi-Synced Digital Clock with TFT Display **Platform:** Arduino (UNO R4 WiFi / compatible board using WiFiS3) **File:** `Clock_Project_v2.ino` **Development period:** March 2026 – April 2026 **Status:** Complete

1. Overview

This project is a self-updating digital clock built around a TFT display shield. It connects to WiFi, pulls accurate time from an NTP (Network Time Protocol) server, and converts it to local Texas Central Time — automatically handling the switch between Central Daylight Time (CDT) and Central Standard Time (CST). The clock keeps drift-free time between syncs, re-syncs itself once an hour, and shows live WiFi/NTP/timezone status on screen.

The display shows:

- Time in 12-hour format (HH:MM:SS) with a blinking colon
 - AM/PM indicator
 - Full date (DD MON YYYY) and day of the week
 - A status bar showing WiFi connection strength, NTP sync state, and current timezone (CDT/CST)
-

2. Hardware

Component	Details
Microcontroller	Arduino board with WiFiS3 support (e.g., UNO R4 WiFi)
Display	DIYables TFT ILI9486 Shield (480×320, landscape)
Connectivity	Built-in WiFi (WiFiS3 library)

3. Software Dependencies

The sketch relies on the following Arduino libraries:

Library	Purpose
DIYables_TFT_Shield	Driver for the ILI9486 TFT display
TimeLib	Core time-keeping functions (hour, minute, day, etc.)
WiFiS3	WiFi connectivity for supported Arduino boards
NTPClient	Fetches current UTC time from an NTP server
WiFiUdp	UDP transport required by NTPClient
Timezone	Converts UTC to local time, including automatic DST rule handling

All of these must be installed via the Arduino Library Manager before compiling.

4. Configuration

Near the top of the file, a configuration block controls network and timezone behavior:

```
const char* ssid    = "YOUR_WIFI_SSID";
```

```
const char* password = "YOUR_WIFI_PASSWORD";
```

```
TimeChangeRule cdt = {"CDT", Second, Sun, Mar, 2, -300}; // UTC-5 (Daylight)
```

```
TimeChangeRule cst = {"CST", First, Sun, Nov, 2, -360}; // UTC-6 (Standard)
```

```
Timezone texasTZ(cdt, cst);
```

```
NTPClient timeClient(ntpUDP, "pool.ntp.org", 0, 60000);
```

 **Security note:** The original sketch has real WiFi credentials hard-coded (`ssid/password`). Never share or commit this file publicly without replacing them — anyone with the file gets your WiFi password. Consider moving credentials to a separate untracked `secrets.h` file if this goes into version control.

To adapt this clock for a different timezone/region, just change the `TimeChangeRule` definitions (rule name, week/day/month of transition, and UTC offset in minutes).

5. How It Works

5.1 Startup Sequence (`setup()`)

1. Initializes serial output (115200 baud) for debug/command use.
2. Initializes the TFT display and draws the static background (borders, boxes, header).
3. Connects to WiFi (`connectToWiFi()`), waiting up to ~15 seconds.
4. Starts the NTP client and performs an initial time sync if WiFi connected.
5. If WiFi/NTP is unavailable, falls back to a hardcoded default time (`Jan 1, 2025, 12:00:00`) so the clock still runs.
6. Draws the initial display.

5.2 Main Loop (`loop()`)

The loop is non-blocking and timer-based (no `delay()` calls), so the display and serial commands stay responsive. It handles four independent timers:

Timer	Interval	Action
Second tick	1000 ms	Calls <code>updateDisplay()</code> to refresh time/date fields
Colon blink	750 ms	Toggles the blinking colon between HH and MM
NTP re-sync	1 hour	Re-syncs time from NTP if already synced
NTP retry	5 minutes	Retries sync if not yet synced (up to 3 attempts per try)

It also checks for incoming serial commands every loop iteration.

5.3 Display Update Strategy

`updateDisplay()` is optimized to avoid full-screen redraws — it tracks the last-drawn value for each field (hour, minute, second, AM/PM, date, WiFi/NTP/DST status) and **only clears and redraws the specific region that changed**. This reduces flicker and keeps the screen responsive even at 1-second refresh rates.

5.4 Time Sync (`syncTimeFromNTP()`)

- Requests the current UTC time from pool.ntp.org, retrying up to 3 times.
- Sanity-checks the returned year (rejects anything outside 2024–2035) to guard against corrupted NTP responses.
- Converts UTC to local Texas time via the [Timezone](#) library, which also determines whether CDT or CST is currently active.
- Re-anchors the internal 1-second display timer immediately after every sync, so the visible seconds never skip or jump.

5.5 Serial Commands

While connected via USB/Serial monitor, the clock accepts single-character commands:

Command	Action
<code>S</code>	Force an immediate NTP re-sync
<code>W</code>	Print current WiFi status, IP, and signal strength (RSSI)
<code>T</code>	Print current time, date, and CDT/CST status
<code>H</code>	Print list of available commands

6. Code Structure Reference

Function	Responsibility
<code>setup()</code>	One-time initialization: display, WiFi, NTP, initial draw
<code>loop()</code>	Main timing loop: display refresh, colon blink, re-sync scheduling, serial input

Function	Responsibility
<code>drawStaticBackground()</code>	Draws the unchanging UI chrome (borders, title, boxes) once at startup
<code>updateDisplay()</code>	Refreshes only the time/date/status fields that have changed
<code>drawColon()</code>	Handles the blinking colon between hours and minutes
<code>drawStatusBar()</code>	Renders WiFi / NTP / DST status at the bottom of the screen
<code>connectToWiFi()</code>	Connects to the configured WiFi network
<code>syncTimeFromNTP()</code>	Fetches UTC time, converts to local time, updates internal clock
<code>processSerialCommand()</code>	Parses and executes single-character serial commands
<code>clearAt()</code>	Helper to clear a rectangular region before redrawing

7. Design Notes & Reliability Features

- **Drift-free timing:** Rather than using a simple `delay(1000)`, the loop compares elapsed `millis()` against a reference timestamp, so the display advances in real elapsed time regardless of how long other operations take.
 - **Graceful degradation:** If WiFi or NTP is unavailable, the clock still runs from a fallback local time rather than freezing or crashing.
 - **Partial redraws only:** Every display field is redrawn only when its value actually changes, minimizing TFT flicker and SPI bus traffic.
 - **Automatic DST handling:** Uses the `Timezone` library's rule-based DST calculation rather than hardcoded dates, so it stays correct year over year.
-

8. Known Limitations / Possible Future Improvements

- WiFi credentials are hardcoded in the sketch (see security note above).

- No on-device WiFi configuration (e.g., WiFiManager captive portal) — SSID/password changes require re-flashing.
 - Timezone is hardcoded to Texas Central Time; not user-configurable without editing source.
 - No persistent storage (e.g., EEPROM/RTC backup) — if power is lost and WiFi is unavailable on reboot, the clock falls back to a fixed default date/time until it can re-sync.
 - Fallback date (Jan 1, 2025) is a placeholder and will display incorrect date info until NTP sync succeeds.
-

9. Project Timeline

Phase	Period
Development	March 2026
Refinement / testing	April 2026
Completion	April 2026